

Spis treści

1	Opis implementacji	3
1.1	Środowisko sprzętowe i wspólne założenia	3
1.2	Iloczyn skalarny (<code>dot</code>)	4
1.2.1	Wariant naiwny (<code>dot_naive</code>)	4
1.2.2	Wariant SIMD (<code>dot_simd</code>)	5
1.2.3	Wariant z wieloma akumulatorami (<code>dot_simd_multiacc</code>)	5
1.2.4	Wariant wielowątkowy (<code>dot_omp</code>)	6
1.3	Mnożenie macierz–wektor (<code>gemv</code>)	6
1.3.1	Wariant naiwny (<code>gemv_naive</code>)	6
1.3.2	Wariant SIMD (<code>gemv_simd</code>)	6
1.3.3	Wariant z prefetchingiem (<code>gemv_simd_prefetch</code>)	6
1.3.4	Wariant blokowany na cache (<code>gemv_avx_fma_blocked</code>)	7
1.3.5	Warianty z mikrojądrem 4-wierszowym (<code>gemv_avx_fma_v2</code> , <code>_v3</code>)	7
1.3.6	Wariant wielowątkowy (<code>gemv_avx_fma_v3_omp</code>)	8
1.4	Mnożenie macierz–macierz (<code>gemm</code>)	8
1.4.1	Wariant naiwny (<code>gemm_naive</code>)	8
1.4.2	Zmiana kolejności pętli (<code>gemm_ikj</code>)	8
1.4.3	Blokowanie trójpoziomowe (<code>gemm_blocked</code>)	9
1.4.4	Pakowane mikrojądro 6×8 (<code>gemm_packed</code> , <code>gemm_omp</code>)	9
1.4.5	Pakowane mikrojądro 4×12 w asemblerze (<code>gemm_zen3</code>)	9
1.4.6	Ścieżka małych macierzy (<code>gemm_zen3_tiny</code>)	10
1.4.7	Współdzielony panel B (<code>gemm_zen3_par</code>)	11
1.4.8	Warianty pośrednie (<code>gemm_zen3_tuned</code> , <code>gemm_zen3_sched</code>)	11
1.4.9	Dyspozytor produkcyjny (<code>gemm_zen3_best</code>)	12
1.4.10	Algorytm Strassena (<code>gemm_strassen</code>)	12
1.5	Splot dwuwymiarowy (<code>conv</code>)	12
1.5.1	Warianty naiwny, z kolejnością pętli i blokowany	13
1.5.2	Splot pakowany (<code>conv_packed</code> , <code>conv_omp</code>)	13
1.5.3	Splot 1×1 jako <code>gemm</code> (<code>conv_1x1</code>)	13

1.5.4	Splot Winograda $F(2 \times 2, 3 \times 3)$ (<code>conv_winograd</code>)	14
1.5.5	Splot blokowany kanałowo NCHWc (<code>conv_nchw</code>)	14
1.5.6	Dyspozytor splotu (<code>conv_zen3</code>)	15
1.5.7	Ścieżki im2col oraz biblioteki wzorcowe	15
1.6	Zestawienie stałych projektowych	16

Rozdział 1

Opis implementacji

W niniejszym rozdziale opisano wszystkie autorskie implementacje czterech jąder obliczeniowych algebry liniowej zaimplementowanych w ramach pracy: iloczynu skalarnego (`dot`), mnożenia macierzy przez wektor (`gemv`), mnożenia macierzy przez macierz (`gemm`) oraz splotu dwuwymiarowego (`conv`). Każde jądro występuje w wielu wariantach, uporządkowanych od implementacji naiwnej (referencyjnej) do w pełni zoptymalizowanej; wariantom autorskim towarzyszą wywołania bibliotek wzorcowych (OpenBLAS, BLIS, libxsmm) służące za punkt odniesienia. Dla każdej implementacji podano zwięzły opis zastosowanej techniki oraz — co jest celem tego rozdziału — wypunktowane uzasadnienie każdej podjętej decyzji projektowej (dobór rozmiarów kafelków, faktorów blokowania, alokacji rejestrów, układu pakowania danych, strategii zrównoleglenia, warunków przełączania ścieżek czy decyzji o użyciu wstawek asemblerowych).

Wszystkie konkretne stałe (`MR_Z`, `KC_Z`, `CB` itd.) cytowane są bezpośrednio z kodu źródłowego. Znaczniki postaci `[CYT: klucz]` oznaczają miejsca, w których w ostatecznej wersji pracy powinny pojawić się odnośniki bibliograficzne; same wpisy bibliograficzne nie są tu jeszcze zamieszczane.

1.1 Środowisko sprzętowe i wspólne założenia

Wszystkie jądra projektowano pod kątem jednego, konkretnego mikroprocesora — AMD Ryzen 5 5600 (mikroarchitektura Zen 3). Decyzje projektowe niższego poziomu wynikają wprost z właściwości tej architektury:

- **Rejestry wektorowe.** Zen 3 udostępnia 16 architektonicznych rejestrów YMM o szerokości 256 bitów (4 liczby `double` lub 8 liczb `float`)^[CYT: agner-microarch]. Liczba ta jest twardym ograniczeniem decydującym o maksymalnym rozmiarze

kafelka mikrojądra — wykorzystywaną wielokrotnie w doborze kształtów 4×12 (`gemm`) oraz 6×16 (`conv`).

- **Jednostki FMA.** Architektura wykonuje instrukcję `vfmadd...` z przepustowością dwóch operacji na takt i opóźnieniem rzędu czterech taktów^[CYT: agner-instr]. Aby ukryć to opóźnienie, mikrojądra utrzymują kilka niezależnych łańcuchów akumulacji (np. 8 akumulatorów w iloczynie skalarnym, 12 w `gemm`).
- **Hierarchia pamięci podręcznej.** L1d = 32 KiB, L2 = 512 KiB, L3 = 32 MiB (współdzielona w obrębie CCX)^[CYT: amd-sog]. Wartości te wyznaczają faktory blokowania KC, MC i NC, tak aby kolejne panele danych rezydowały na właściwych poziomach hierarchii.
- **Przepustowość pamięci.** Zmierzone przepustowości efektywne (DDR4-3200 oraz poszczególne poziomy cache) stanowiące górne ograniczenie modelu *roofline* przejęto z wcześniejszych pomiarów autora dotyczących jądra `gemv`^[CYT: gemv-paper].

Wspólnym elementem niemal wszystkich jąder jest *redukcja pozioma* rejestru wektorowego do skalar (funkcja `hsum_pd`) oraz obsługa skalnych „ogonów” (elementów niemieszczących się w pełnym kafelku wektorowym). Atrybuty `__attribute__((hot))` oraz `always_inline` stosowane są do wymuszenia agresywnego inliningu mikrojąder.

1.2 Iloczyn skalarny (dot)

Iloczyn skalarny dwóch wektorów $a, b \in \mathbb{R}^n$,

$$s = \sum_{i=0}^{n-1} a_i b_i, \quad (1.1)$$

jest operacją o bardzo niskiej intensywności arytmetycznej (jedna operacja FMA na dwa odczyty z pamięci), więc o jego wydajności decyduje przepustowość pamięci oraz zdolność do ukrycia opóźnienia FMA. Kolejne warianty: `naive` $\rightarrow$ `simd` $\rightarrow$ `simd_multiacc` $\rightarrow$ `omp`.

1.2.1 Wariant naiwny (dot_naive)

Pojedyncza pętla skalarna sumująca $a_i b_i$ do jednego akumulatora. Stanowi punkt odniesienia mierzący narzut samego kompilatora i opóźnienie pamięci bez jawnej wektoryzacji.

1.2.2 Wariant SIMD (dot_simd)

Wektoryzacja AVX2: pętla przetwarza po 4 liczby `double` na iterację (`_mm256_fmadd_pd`) do jednego akumulatora `__m256d`, a na końcu wykonuje jedną redukcję poziomą.

- **Pojedynczy akumulator** minimalizuje presję na rejestry, ale tworzy jeden łańcuch zależności FMA — przy krótkich wektorach opóźnienie FMA nie jest ukryte, co jest świadomie zaakceptowanym ograniczeniem tego wariantu.
- **Skalarny ogon** obsługuje elementy spoza wielokrotności 4.

1.2.3 Wariant z wieloma akumulatorami (dot_simd_multiacc)

Kluczowy wariant zoptymalizowany: 8 niezależnych akumulatorów (`s0...s7`), pętla rozwinięta o czynnik 32 (8 akumulatorów $\times$ 4 liczby), a następnie redukcja drzewiasta. Implementację przedstawia Listing 1.1.

- **Osiem łańcuchów akumulacji** całkowicie ukrywa czterotaktowe opóźnienie FMA: przy przepustowości dwóch FMA na takt potrzeba co najmniej ośmiu niezależnych operacji „w locie”^[CYT: agner-instr].
- **Rozwinięcie o 32** dobrano tak, aby 8 akumulatorów (`ymm`-y wyniku) wraz z rejestrami odczytu wypełniało plik rejestrów bez spillowania.
- **Redukcja drzewiasta** ($t_0=s_0+s_1, \dots$, następnie sumy częściowe) skraca ścieżkę krytyczną redukcji końcowej względem łańcucha liniowego.

Listing 1.1: Jądro iloczynu skalarnego z ośmioma niezależnymi akumulatorami (`dot_simd_multiacc.c`).

```

1  __m256d s0=_mm256_setzero_pd(), s1=_mm256_setzero_pd(), /* ... */ s7=_mm256_setzero_pd();
2  for (; i + 31 < n; i += 32) {
3      s0 = _mm256_fmadd_pd(_mm256_loadu_pd(&a[i+ 0]), _mm256_loadu_pd(&b[i+ 0]), s0);
4      s1 = _mm256_fmadd_pd(_mm256_loadu_pd(&a[i+ 4]), _mm256_loadu_pd(&b[i+ 4]), s1);
5      /* ... s2..s6 ... */
6      s7 = _mm256_fmadd_pd(_mm256_loadu_pd(&a[i+28]), _mm256_loadu_pd(&b[i+28]), s7);
7  }
8  __m256d t0=_mm256_add_pd(s0,s1), t1=_mm256_add_pd(s2,s3);
9  __m256d t2=_mm256_add_pd(s4,s5), t3=_mm256_add_pd(s6,s7);
10 __m256d acc=_mm256_add_pd(_mm256_add_pd(t0,t1), _mm256_add_pd(t2,t3));

```

1.2.4 Wariant wielowątkowy (dot_omp)

Zrównoleglenie przez `#pragma omp parallel reduction(+:total)`: wektor jest dzielony na równe fragmenty, a każdy wątek wywołuje na swoim fragmencie zoptymalizowane jądro `dot_simd_multiacc`.

- **Ponowne użycie jądra skalarnego SIMD** zamiast osobnego kodu wielowątkowego — każdy wątek korzysta z w pełni zoptymalizowanej ścieżki.
- **Redukcja OpenMP** sumuje wyniki częściowe wątków bez jawnej synchronizacji.

1.3 Mnożenie macierz–wektor (gemv)

Operacja `gemv` realizuje

$$y \leftarrow \alpha Ax + \beta y, \quad A \in \mathbb{R}^{m \times n}, \quad (1.2)$$

w układzie wierszowym (*row-major*). Podobnie jak iloczyn skalarny jest ograniczona przepustowością pamięci, dlatego optymalizacje koncentrują się na ponownym wykorzystaniu wczytanego wektora x przez wiele wierszy jednocześnie oraz na dopasowaniu rozmiarów bloków do hierarchii cache.

1.3.1 Wariant naiwny (gemv_naive)

Podwójna pętla: dla każdego wiersza sumowane są iloczyny $A_{ij}x_j$, po czym zapisywane jest $y_i = \alpha \text{sum} + \beta y_i$. Implementacja referencyjna.

1.3.2 Wariant SIMD (gemv_simd)

Na wiersz przypadają dwa akumulatory YMM przetwarzające 8 kolumn na iterację (dwa bloki po 4). Obsługa β : `memset` dla $\beta = 0$ lub mnożenie skalarne dla $\beta \neq 1$.

- **Dwa akumulatory** częściowo rozdzielają zależności w ramach jednego wiersza.
- **Redukcja pozioma raz na wiersz**, po zakończeniu pętli kolumn.

1.3.3 Wariant z prefetchingiem (gemv_simd_prefetch)

Rozszerza `gemv_simd` o programowy prefetching. Stała `PREFETCH_DIST = 8`.

- **Prefetch wyprzedzający wiersz** `A[(i+8)*cols]` ukrywa opóźnienie prowadzenia kolejnego wiersza z L3/pamięci.

- **Prefetch wewnątrz wiersza** pod adresem +64 bajty nakłada sprowadzanie danych na bieżące obliczenia.
- **Odczyty wyrównane** (`_mm256_load_pd`) zamiast niewyrównanych — przy założeniu wyrównania bufora do 32 bajtów.

1.3.4 Wariant blokowany na cache (`gemv_avx_fma_blocked`)

Trzy warianty mikrojądra (L1/L2/L3) wybierane na podstawie rozmiaru zbioru roboczego. Progi: `L1_THRESHOLD` = 24 KB (75 % z 32 KB), `L2_THRESHOLD` = 384 KB (75 % z 512 KB). Bloki dobrano w osobnym etapie strojenia (`tuning.c`): L1 i L3 — 64×512 , L2 — 128×256 .

- **Dyspozytor wg rozmiaru** dobiera blok i politykę prefetchingu do poziomu cache mieszczącego zbiór roboczy; jądro L1 nie używa prefetchingu (dane już w cache).
- **Margines 75 %** pozostawia miejsce na wektor x , wektor wynikowy i dane wypierane przez system.
- **Strojenie offline** — konkretne rozmiary bloków są wynikiem przeszukania siatki parametrów programem `tuning.c`, nie zaś analitycznego oszacowania^{[CYT: [gemv-paper](#)]}.
- **Makro** `DEFINE_BLOCKED_KERNEL` generuje trzy warianty jądra z jednego szablonu, eliminując powielanie kodu.

1.3.5 Warianty z mikrojądrem 4-wierszowym (`gemv_avx_fma_v2`, `_v3`)

Mikrojądro przetwarza jednocześnie 4 wiersze macierzy, mnożąc je przez wspólnie wczytany fragment wektora x . Wariant `v2` używa jednego akumulatora na wiersz; wariant `v3` — dwóch akumulatorów na wiersz i rozwinięcia o 16 kolumn.

- **Współdzielenie wektora** x przez 4 wiersze czterokrotnie zwiększa intensywność arytmetyczną względem przetwarzania wiersz po wierszu (jeden odczyt x_j obsługuje cztery operacje FMA).
- **Podwójne akumulatory w v3** (8 akumulatorów: 2 na wiersz) rozdzielają zależności FMA przy rozwinięciu o 16 kolumn, kosztem większej liczby zajętych rejestrów.
- **Jedna redukcja pozioma na wiersz** pozostaje mimo rozwinięcia, dzięki scaleniu akumulatorów częściowych dopiero przed redukcją.

1.3.6 Wariant wielowątkowy (gemv_avx_fma_v3_omp)

Wielowątkowa wersja v3 z progami uruchamiania zrównoleglenia.

- **Progi.** `MIN_ROWS_FOR_PARALLEL = 128`, `MIN_ELEMENTS_FOR_PARALLEL = 256 × 256`, `MAX_ELEMENTS_FOR_PARALLEL = 8 M`. Poniżej dolnych progów dominuje narzut tworzenia wątków; powyżej górnego progu problem jest ograniczony przepustowością pamięci i wątki rywalizują o kontroler pamięci, więc zrównoleglenie nie przynosi korzyści^[CYT: gemv-paper].
- **Pojedynczy region** `#pragma omp parallel` z ręcznym podziałem wierszy zamiast `omp parallel for` — eliminuje narzut harmonogramowania na iterację.
- **Granice wątków wyrównane do 4 wierszy**, aby każdy fragment używał 4-wierszowego mikrojądra SIMD.

1.4 Mnożenie macierz–macierz (gemm)

Operacja `gemm` realizuje

$$C \leftarrow \alpha A B + \beta C, \quad A \in \mathbb{R}^{M \times K}, B \in \mathbb{R}^{K \times N}, C \in \mathbb{R}^{M \times N}. \quad (1.3)$$

W odróżnieniu od poprzednich jąder `gemm` jest ograniczona mocą obliczeniową (intensywność arytmetyczna rośnie z rozmiarem), więc kluczowe stają się: kształt mikrojądra maksymalizujący wykorzystanie jednostek FMA, pakowanie danych zapewniające sekwencyjny dostęp oraz wielopoziomowe blokowanie na cache. Struktura pętli i pakowania jest wzorowana na kanonicznej „anatomii” wysokowydajnego `gemm`^[CYT: goto2008].

1.4.1 Wariant naiwny (gemm_naive)

Trzy zagnieżdżone pętle w kolejności `i, j, k` z akumulacją iloczynu wewnętrznego. Złożoność $O(N^3)$ przy bardzo słabej lokalności (kolumnowy odczyt macierzy B).

1.4.2 Zmiana kolejności pętli (gemm_ikj)

Kolejność `i, k, j` z wyniesieniem αA_{ik} przed pętlę po j . Wiersz $B_{k,:}$ jest wówczas odczytywany sekwencyjnie.

- **Sekwencyjny dostęp do B** poprawia wykorzystanie linii cache względem wariantu naiwnego.
- **Wyniesienie mnożenia przez α** — raz na iterację k zamiast raz na parę (i, j) .

1.4.3 Blokowanie trójpoziomowe (gemm_blocked)

Blokowanie z rozmiarami $BM = 64$, $BN = 64$, $BK = 256$ (kolejność pętli j, c, i a następnie i, k, j).

- **Blok A o rozmiarze 64×256 (≈ 128 KiB)** i analogiczny blok B projektowane są pod rezydencję w L2.
- **Brak pakowania** — wariant pokazuje zysk z samego blokowania, jeszcze bez kosztu i korzyści przepakowania danych.

1.4.4 Pakowane mikrojądro 6×8 (gemm_packed, gemm_omp)

Pierwsza implementacja w pełni zgodna ze schematem GotoBLAS/BLIS: dane A i B są przepakowywane do paneli o sekwencyjnym układzie, a obliczenia wykonuje mikrojądro 6×8 (Listing 1.2). Stałe: $MR = 6$, $NR = 8$, $MC = 72$, $KC = 256$, $NC = 4080$.

- **Kafelek 6×8** wykorzystuje 12 akumulatorów (6 wierszy $\times$ 2 pasy po 4 liczby double) plus 1 rozgłoszenie A i 2 wektory B — razem 15 z 16 rejestrów YMM; jest to kanoniczny kafelek BLIS dla architektur Haswell/Zen^[CYT: blis2015].
- **Układ pakowania.** Panel A ma rozstaw MR między wierszami jednej kolumny k , co pozwala mikrojądru pobierać a_i instrukcją `vbroadcastsd` z pamięci ciągłej; panel B ma rozstaw NR ^[CYT: goto2008].
- **Epilog** `vfmadd213pd` liczy $C += \alpha \cdot \text{acc}$ w miejscu, łącząc mnożenie przez α z dodaniem starej wartości C .
- **Obsługa krawędzi** przez skalarne jądro `ukr_6x8_edge` dla niepełnych kafelków.
- **Wariant wielowątkowy (gemm_omp)** dzieli wymiar M między wątki (zaokrąglając do MR); każdy wątek pakuje własną kopię B — jest to najprostsza, lecz pamięciowo kosztowna strategia, stanowiąca punkt odniesienia dla wariantów ze współdzielonym B .

1.4.5 Pakowane mikrojądro 4×12 w assemblerze (gemm_zen3)

Mikrojądro dostrojone specjalnie do Zen 3, z pętlą K zapisaną we wstawce assembly (Listing 1.3). Stałe: $MR_Z = 4$, $NR_Z = 12$, $MC_Z = 192$, $KC_Z = 240$, $NC_Z = 4080$.

- **Kafelek 4×12 .** 12 akumulatorów (4 wiersze $\times$ 3 pasy) + 1 rozgłoszenie A + 3 wektory B wykorzystuje *dokładnie* 16 rejestrów YMM. Analiza zliczania μ -operacji wskazuje, że ten kształt daje korzystniejszy stosunek odczytów do operacji FMA niż kanoniczny 6×8 przy pełnym wysyceniu pliku rejestrów^[CYT: agner-instr].

Listing 1.2: Pętla K mikrojądra 6×8 z 12 akumulatorami (`gemm_packed.c`).

```

1  for (int k = 0; k < kc; k++) {
2      __m256d b0 = _mm256_load_pd(&B_pack[k*NR + 0]);
3      __m256d b1 = _mm256_load_pd(&B_pack[k*NR + 4]);
4      __m256d va;
5      va = _mm256_broadcast_sd(&A_pack[k*MR + 0]);
6      c00 = _mm256_fmadd_pd(va, b0, c00); c01 = _mm256_fmadd_pd(va, b1, c01);
7      /* ... wiersze 1..4 ... */
8      va = _mm256_broadcast_sd(&A_pack[k*MR + 5]);
9      c50 = _mm256_fmadd_pd(va, b0, c50); c51 = _mm256_fmadd_pd(va, b1, c51);
10 }

```

- **Wstawka asemblerowa** jest konieczna, ponieważ przy pełnym wysyceniu 16 rejestrów kompilator GCC wprowadza zmienne tymczasowe i przepełnia (*spilluje*) dwa akumulatory na stos; jawne przypięcie akumulatorów do ymm4–ymm15 eliminuje ten problem (szczegóły w dokumencie `INLINE_ASM_RATIONALE.md`)^[CYT: agner-microarch].
- **Brak programowego prefetchingu w pętli K .** Plik rejestrów jest pełny, a obliczanie adresu prefetcha wymusiłoby spill akumulatorów; sprzętowy prefetcher strumieniowy Zen 3 jest dla odczytów sekwencyjnych wystarczający i zalecany^[CYT: agner-microarch].
- **Dobór faktorów blokowania.** $KC_Z = 240$ dobrano tak, aby mikropanel A ($\approx 7,5$ KiB) mieścił się w L1d wraz z rezydentnym fragmentem B ; $MC_Z = 192$ wynika z analitycznego ograniczenia rezydencji panelu w L2^[CYT: goto2008]; $NC_Z = 4080$ (panel $\approx 7,5$ MiB) odpowiada rezydencji w L3 współdzielonej między rdzeniami CCX^[CYT: amd-sog].
- **Pakery wektorowe** (plik `gemm_zen3_pack.h`) realizują transpozycję bloków 4×4 macierzy A instrukcjami `unpack/permute2f128` oraz trzy ciągle kopie 256-bitowe na wiersz B ; nagłówek jest współdzielony przez kilka jednostek translacji, aby każda otrzymała własną wersję inline.

1.4.6 Ścieżka małych macierzy (`gemm_zen3_tiny`)

Dla $\min(M, N, K) \leq 96$ mikrojądro 4×12 działa bezpośrednio na wejściowych macierzach wierszowych, bez pakowania i bez `aligned_alloc`.

- **Pominięcie trzech narzutów** (alokacja, pakowanie, osobny przebieg `apply_beta`), które dominują przy rozmiarach rzędu mikrosekund; wzorowane na szybkich ścieżkach małych macierzy w BLIS^[CYT: blis2015].

Listing 1.3: Fragment pętli K mikrojądra 4×12 we wstawce assemblerowej; akumulatory są przypięte do ymm4–ymm15 (gemm_zen3.c).

```

1  ".balign 32                                \n"
2  "1:                                        \n"
3  "vmovapd    0(%[B]), %%ymm1                \n"    // 3 wektory B (12 kolumn)
4  "vmovapd   32(%[B]), %%ymm2                \n"
5  "vmovapd   64(%[B]), %%ymm3                \n"
6  "vbroadcastsd 0(%[A]), %%ymm0              \n"    // rozgłoszenie A[0]
7  "vfmadd231pd %%ymm1, %%ymm0, %%ymm4        \n"
8  "vfmadd231pd %%ymm2, %%ymm0, %%ymm5        \n"
9  "vfmadd231pd %%ymm3, %%ymm0, %%ymm6        \n"
10 /* ... wiersze 1..3 -> ymm7..ymm15 ... */
11 "addq $32, %[A]                            \n"
12 "addq $96, %[B]                            \n"
13 "decq %[k]                                \n"
14 "jnz 1b                                    \n"

```

- **Dodatkowe jądro** 4×4 wektoryzuje częsty przypadek $N \bmod 12 \in \{4, 8\}$ (np. $N = 64$, $N = 256$), zastępując kosztowną pętlę skalarną.
- **Epilog „tylko zapis”** dla $\beta = 0$ pomija odczyt C .

1.4.7 Współdzielony panel B (gemm_zen3_par)

Edukacyjny wariant wielowątkowy izolujący jeden zysk: współdzielenie panelu B .

- **Jeden wspólny B_{pack}** dla wszystkich wątków ($\approx 7,5$ MiB) zamiast kopii na wątek, dzięki czemu panel mieści się w L3 raz, a nie wielokrotnie; każdy wątek pakuje własny panel A .
- **Zrównoleglenie pętli ic** (styl wielopętlowy BLIS) — pętłe jc i pc pozostają sekwencyjne^[CYT: blis2015].

1.4.8 Warianty pośrednie (gemm_zen3_tuned, gemm_zen3_sched)

- **gemm_zen3_tuned** dodaje trwały (*persistent*) bufor roboczy reużywany między wywołaniami oraz wektorowe pakery, lecz bez dyspozytora rozmiarów — izoluje wpływ pojedynczej optymalizacji.
- **gemm_zen3_sched** stosuje jawne harmonogramowanie: wektor B dla iteracji $k+1$ jest wczytywany na końcu iteracji k , aby ukryć pięciotaktowe opóźnienie odczytu z L1 pod operacjami FMA poprzedniej iteracji (rozwiniecie pętli K o 4).

1.4.9 Dyspozytor produkcyjny (gemm_zen3_best)

Łączy wszystkie skuteczne optymalizacje pod jednym dyspozytorem.

- **Trójdrożny wybór ścieżki.** (i) $\min(M, N, K) \leq 96$ — ścieżka „tiny”; (ii) dostatecznie wiele bloków ic ($M \geq n_t \cdot \text{MC_Z}$) — współdzielony B z równoległą pętlą ic; (iii) w przeciwnym razie — podział wymiaru M między wątki (osobne B), gdyż przy małym M współdzielony B pozostawiłby część wątków bezczynnych.
- **Równoległe pakowanie B** włączane tylko, gdy liczba pasów jest dostatecznie duża, aby zamortyzować barierę OpenMP ($\approx 30 \mu s$ przy 6 wątkach).
- **Trwały bufor roboczy** eliminuje koszt `aligned_alloc` na każde wywołanie [\[CYT: blis2015\]](#).

1.4.10 Algorytm Strassena (gemm_strassen)

Wariant wykorzystujący schemat Strassena–Winograda o siedmiu mnożeniach na rozkładzie ćwiartkowym $A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}$ (analogicznie B, C). Zamiast ośmiu mnożeń klasycznych wykonuje się siedem, kosztem dodatkowych przejść dodawania/odejmowania macierzy.

- **Próg STRASSEN_MIN = 2048.** Poniżej tego rozmiaru 12,5% oszczędności mnożeń nie równoważy kosztu osiemnastu przejść `mat_add/mat_sub/mat_copy`; próg wyznaczono empirycznie na Ryzen 5 5600 [\[CYT: schwartz2023\]](#).
- **Przypadek bazowy** (poniżej progu) deleguje do `gemm_zen3_best_omp`.
- **Zrównoleglone przejścia macierzowe.** Wszystkie 18 przejść jest zrównoleglonych po wymiarze wierszy (`#pragma omp parallel for`); przy 4096^3 koszt pojedynczego przejścia spada z ~ 6 ms (1 wątek) do ~ 1 ms (6 rdzeni).
- **Kopiowanie ćwiartek przez bufor S_1/S_2 .** Mikrojądro `gemm_zen3` zakłada, że rozstaw wiersza równa się wymiarowi macierzy (brak parametrów `lda/lb/lc`); surowe ćwiartki dziedziczą rozstaw macierzy rodzica, więc przed każdym wywołaniem są kopiowane do ciągłych buforów.

1.5 Splot dwuwymiarowy (conv)

Splot realizowany jest dla pojedynczej próbki, kroku 1, bez dopełnienia, w typie `float32`, dla danych w układzie NCHW:

$$Y_{o,i,j} = \sum_{c=0}^{C_{in}-1} \sum_{p=0}^{K_H-1} \sum_{q=0}^{K_W-1} X_{c,i+p,j+q} W_{o,c,p,q}, \quad (1.4)$$

gdzie $X \in \mathbb{R}^{C_{in} \times H \times W}$, $W \in \mathbb{R}^{C_{out} \times C_{in} \times K_H \times K_W}$, $O_H = H - K_H + 1$, $O_W = W - K_W + 1$. Charakter obliczeń silnie zależy od kształtu jądra, dlatego — inaczej niż w `gemm` — istnieje kilka jakościowo różnych ścieżek (splot bezpośredni, 1×1 jako `gemm`, Winograd) wybieranych przez dyspozytor.

1.5.1 Warianty naiwny, z kolejnością pętli i blokowany

- `conv_naive` — sześć zagnieżdżonych pętli, implementacja referencyjna o słabej lokalności.
- `conv_reorder` — kolejność `oc, ic, kh, kw, oh, ow`; dla ustalonego (oc, ic, kh, kw) wewnętrzne pętle czytają X i zapisują Y sekwencyjnie po ow , a waga $W_{oc, ic, kh, kw}$ jest niezmiennikiem wyniesionym przed pętlę.
- `conv_blocked` — blokowanie z `BLOCK_OC = 32`, `BLOCK_OH = 16`, `BLOCK_IC = 32`, ograniczające zbiór roboczy do L1/L2.

1.5.2 Splot pakowany (conv_packed, conv_omp)

Mikrojądro o kafelku `CONV_MR = 6` pozycji przestrzennych $\times$ `CONV_NR = 16` kanałów wyjściowych. Wagi są przepakowywane do układu z C_{out} jako wymiarem najbardziej wewnętrznym.

- **Kafelek 6×16 .** 6 akumulatorów (po jednym na pozycję ow , każdy jako 2 pasy po 8 liczb `float`) + 1 wektor wag + 1 rozgłoszona wartość X — razem 8 aktywnych rejestrów YMM, z zapasem na zmianę kolejności wykonania przez procesor.
- **Przepakowanie wag** (`conv_pack_W_oc_innermost`) tak, aby wartości dla kolejnych kanałów wyjściowych leżały ciągle — mikrojądro rozgłasza wówczas wagę dla 16 kanałów jedną instrukcją.
- **Wariant `conv_omp`** pakuje wagi raz (wspólny `W_pack`), a następnie dzieli C_{out} między wątki (zaokrąglając do `CONV_MR`); każdy wątek pakuje własny fragment X .

1.5.3 Splot 1×1 jako `gemm` (conv_1x1)

Dla jąder $K_H = K_W = 1$ splot jest czystym mnożeniem macierzy $Y(C_{out}, O_H O_W) = W(C_{out}, C_{in}) \cdot X(C_{in}, O_H O_W)$.

- **Brak `im2col`.** W układzie NCHW macierz X jest już ułożona jako $(C_{in}, O_H O_W)$, więc wywołanie kierowane jest bezpośrednio do `cblas_sgemm`, z pominięciem bufora pośredniego.

- **Fallback** do ścieżki pakowanej dla jąder innych niż 1×1 .

1.5.4 Splot Winograda $F(2 \times 2, 3 \times 3)$ (conv_winograd)

Dla jąder 3×3 o kroku 1 zastosowano algorytm Winograda, który redukuje liczbę mnożeń na kafelek wyjściowy 2×2 z 36 (splot bezpośredni) do 16^[CYT: lavin2016]. Wynik kafelka dany jest wzorem

$$Y = A^T \left[(G g G^T) \odot (B^T d B) \right] A, \quad (1.5)$$

gdzie $\odot$ oznacza mnożenie po współrzędnych, g to filtr 3×3 , d — kafelek wejściowy 4×4 , a macierze transformacji to

$$G = \begin{bmatrix} 1 & 0 & 0 \\ \frac{1}{2} & \frac{1}{2} & \frac{1}{2} \\ \frac{1}{2} & -\frac{1}{2} & \frac{1}{2} \\ 0 & 0 & 1 \end{bmatrix}, \quad B^T = \begin{bmatrix} 1 & 0 & -1 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & -1 & 1 & 0 \\ 0 & 1 & 0 & -1 \end{bmatrix}, \quad A^T = \begin{bmatrix} 1 & 1 & 1 & 0 \\ 0 & 1 & -1 & -1 \end{bmatrix}.$$

Obliczenia rozbito na cztery etapy (Listing 1.4): transformacja filtrów U , transformacja wejścia V , 16 mnożeń macierzy $M_i = U_i \cdot V_i$ oraz transformacja wyjścia.

- **Redukcja mnożeń** $36 \rightarrow 16$ ($2, 25\times$) jest źródłem zysku; koszt to cztery transformacje kafelków 4×4 oraz 16 wywołań `sgemm`^[CYT: lavin2016].
- **Warunki stosowalności.** $K_H = K_W = 3$, $O_H O_W \geq 16$ (amortyzacja narzutu transformacji) oraz parzyste O_H i O_W (kafelkowanie 2×2).
- **Macierze o wpisach całkowitych/wymiernych** pozwalają zastąpić mnożenia w transformacjach $B^T d B$ i $A^T M A$ samymi dodawaniami i odejmowaniami^[CYT: castello2023].
- **Obsługa krawędzi.** Przy nieparzystych O_H/O_W ostatni wiersz i kolumna wyniku liczone są bezpośrednio (ułamek pracy rzędu $1/O_H$).

1.5.5 Splot blokowany kanałowo NCHWc (conv_nchw)

Bezpośredni splot na układzie blokowanym kanałowo z $CB = 8$ (jeden pas YMM w float32): X przekształcane jest do $(C_{in}/C_b, H, W, C_b)$, a wagi do $(C_{out}/C_b, C_{in}/C_b, K_H, K_W, C_b, C_b)$.

- $CB = 8$ odpowiada szerokości jednego pasa YMM dla float; mikrojądro wczytuje jeden wektor wag (8 kanałów wyjściowych) i rozgłasza 6 wartości przestrzennych X , akumulując do 6 rejestrów — 8 aktywnych YMM^[CYT: onednn].
- **Brak rozgłaszania po stronie wag** — rozgłaszane są wartości X , co eliminuje dodatkowe instrukcje na pętli wag.
- **Fallback** do ścieżki pakowanej, gdy C_{in} lub C_{out} nie jest wielokrotnością 8.

Listing 1.4: Transformacja wejścia Winograda $v = B^T d B$ rozpisana na dodawania (conv_winograd.c).

```

1  for (int j = 0; j < 4; j++) {           // t = B^T d
2      t[0][j] = d[0][j] - d[2][j];
3      t[1][j] = d[1][j] + d[2][j];
4      t[2][j] = -d[1][j] + d[2][j];
5      t[3][j] = d[1][j] - d[3][j];
6  }
7  for (int i = 0; i < 4; i++) {           // v = t B
8      v[i][0] = t[i][0] - t[i][2];
9      v[i][1] = t[i][1] + t[i][2];
10     v[i][2] = -t[i][1] + t[i][2];
11     v[i][3] = t[i][1] - t[i][3];
12 }

```

1.5.6 Dyspozytor splotu (conv_zen3)

Wybiera ścieżkę specjalizowaną na podstawie kształtu jądra, analogicznie do `gemm_zen3_best`:

- $K_H = K_W = 1 \Rightarrow \text{conv_1x1}$ (SGEMM);
- $K_H = K_W = 3$ oraz $O_H O_W \geq 16$ i parzyste $O_H, O_W \Rightarrow \text{conv_winograd}$;
- w przeciwnym razie $\Rightarrow \text{conv_nchw}$.

Region OpenMP znajduje się wewnątrz funkcji docelowych (w transformacjach układu i pętlach głównych), a nie wokół samego dyspozytora.

1.5.7 Ścieżki im2col oraz biblioteki wzorcowe

- `conv_im2col_openblas` / `conv_im2col_blis` — rozwijają X do macierzy kolumnowej (`im2col`) i wywołują SGEMM. Układ wierszowy $Y = W \cdot \text{col}$ uzyskuje się sztuczką transpozycji: w ujęciu kolumnowym (Fortran) operandy są oglądane jako (N, K) i (K, C_{out}) , co daje ten sam układ bajtów co pożądane mnożenie wierszowe.
- `conv_libxsmm` — cienka nakładka na JIT-owane SGEMM biblioteki `libxsmm`, ładowanej dynamicznie; przy braku biblioteki funkcja jest pusta (pozycja w benchmarku oznaczana jako N/A). Wykorzystano publiczne wejście `libxsmm_sgemm` ze względu na stabilność API między wersjami^[CYT: libxsmm2016].

Tabela 1.1: Zestawienie kluczowych stałych projektowych jąder

Jądro / wariant	Stała	Uzasadnienie
<code>dot_simd_multiacc</code>	8 akumul., unroll 32	ukrycie opóźnień FMA
<code>gemv</code> blokowany	progi 24 KB / 384 KB	75 % pojemności L1 / L2
<code>gemm</code> 6×8	MR=6, NR=8, MC=72, KC=256, NC=4080	kanoniczny kafelek BLIS, rezydencja L1/L2/L3
<code>gemm</code> 4×12	MR=4, NR=12, MC=192, KC=240, NC=4080	pełne wysycenie 16 YMM, dostrojenie do Zen 3
<code>gemm_strassen</code>	STRASSEN_MIN=2048	empiryczny próg opłacalności
<code>conv</code> pakowany	CONV_MR=6, CONV_NR=16	8 aktywnych YMM
<code>conv_nchw</code>	CB=8	jeden pas YMM (float32)
<code>conv_winograd</code>	$F(2 \times 2, 3 \times 3)$	redukcja mnożeń $36 \rightarrow 16$

1.6 Zestawienie stałych projektowych

Tabela 1.1 zestawia kluczowe stałe kafeleków i blokowania poszczególnych jąder wraz z ich uzasadnieniem sprzętowym.